

ClaimDesk QA

An auto

What it is, what it proves, how ready it is, and how to run it. Written for someone who has never seen the code.

1. What this project is

Every time a team changes their software, someone has to check that nothing else broke. Doing that by hand is slow, boring, and people miss things. So teams write programs that test other programs, automatically, on every change.

This project is one of those. It has two halves, and only the second one is the point.

Half one: the practice dummy

A small insurance-claims website called ClaimDesk. Customers file claims, handlers review them, administrators approve payouts. It is a real working app, not a stub, but it exists only to give the tests something realistic to work on. Think of it as the crash-test dummy, not the car.

Half two: the testing framework (the actual deliverable)

The code that drives ClaimDesk and checks it behaves correctly. This is the part that represents the skill being demonstrated. It is built like a real engineering product: installable, strictly typed, documented, and running automatically on three separate CI systems.

2. How it checks the app: three independent angles

The same application is checked three different ways, because each one can catch what the others cannot.

- Through the browser (32 tests). Drives a real Chrome, Firefox and Safari engine, clicking and typing like a person. Catches anything broken on screen.
- Through the API (157 tests). Talks to the app's data interface directly, skipping the screen. Faster, and catches rule violations the screen might hide.
- Through the database (28 tests). Reads the stored records to confirm what the app said it did is what it actually saved.
- End to end (4 tests). One journey that starts in the browser, passes through the API, and finishes by checking the database.
- The tools themselves (130 tests). The testing framework is also software, so it is tested too. This is unusual and is a large part of what makes the project credible.

351 tests in total, plus 58 the sample application ships with.

3. The one rule that makes it professional

The test code is never allowed to import the application's own code. It may only reach the app the way an outsider would: over the network, or by reading the database.

This matters more than it sounds. If tests can reach inside the app, they start sharing the app's assumptions, and a test that shares a bug's assumption cannot see the bug. Keeping them apart is the difference between testing the product and testing your own opinion of it.

The project enforces this three ways, none of which depend on remembering:

- A linter rule that refuses the import outright and fails the build.
- A database login that holds read permission and nothing else, so a test physically cannot invent data the app would never produce.
- A container image that does not contain the application at all. CI proves it by checking the import fails inside the image.

4. By the numbers

Tests	351 passing (130 tools, 157 API, 32 browser, 28 database, 4 end-to-end)
-------	--

Run time, c

Every number above came from a command that was actually run. Where something was written but never executed, the project's progress log says so explicitly rather than quietly implying success.

5. How ready is it?

Working and proven

OKAll 351 tests pass on a developer machine, inside Docker, and on a real Jenkins controller.

OKGitHub Actions runs the suite on every push. The tests and docker workflows are green.

OKCross-browser (Chromium, Firefox, WebKit) verified green.

OKZero flaky failures measured over ten consecutive parallel runs.

OK114 items in the progress log are marked verified by an observed command.

Known gaps, stated honestly

- ! The nightly job's report-publishing step fails. A shell script is missing its executable permission bit, so the badge shows red. The tests themselves all pass; only the report upload fails. It is a one-line fix and is not yet applied.
- ! 12 items remain marked NOT VERIFIED in the progress log. Most were later superseded by a phase that did verify them, and each says which one.
- ! The automatic retry path has never actually fired, because it only runs in CI and nothing has failed there in

a way that triggers it.

Verdict: ready to show. It is complete, it runs, and it is honest about its own edges, which is itself the point the project is making.

6. How to run it

Two routes. The second needs almost nothing installed.

Route A: on your own machine

You need Python 3.11 to 3.13 and PostgreSQL installed. In PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[dev,app]"

pre-commit install --hook-type pre-commit --hook-type commit-msg

Copy-Item .env.example .env          # then set the two passwords

.\scripts\local_db.ps1 start        # throwaway database, yours untouched
python -m uvicorn claimdesk.main:app --app-dir app --port 8000
```

Leave that running. In a second terminal:

```
pytest -m framework -q      # 130 tests, needs nothing else running
pytest -m api -q            # 157 API tests
pytest -m ui -q             # 32 browser tests
pytest -m db -q             # 28 database checks
pytest -m e2e -q            # 4 full journeys

.\scripts\run_suite.ps1     # everything, parallel then serial
.\scripts\report.ps1        # build and open the visual report
```

Browser tests need their browser downloaded once: playwright install chromium. You can log into the app yourself at <http://127.0.0.1:8000/login> using `customer@example.com` with password `Passw0rd!seed`.

Route B: everything in containers

Needs only Docker. Nothing is installed on your machine.

```
docker compose --profile test -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up -d db sut
docker compose -f docker/docker-compose.yml run --build --rm tests
```

If pip fails with a certificate error

Some PostgreSQL installers point `CURL_CA_BUNDLE` at a file that does not exist. It is an environment problem, not a project one, and it stops the very first step:

```
Remove-Item Env:CURL_CA_BUNDLE -ErrorAction SilentlyContinue
```

7. Where to look in the repository

README.md

Start here. The whole project in one page.

docs/progr

If someone technical only has five minutes, point them at the section of the README titled "What makes this not a toy project". It lists real defects the work uncovered, including a test that passed for the wrong reason, a timezone bug that could only fail for five and a half hours a day, and a 3.5x slowdown that turned out not to be the application's fault.